

Murine antibody humanization via
frequentist analysis, simulated
annealing, and support vector
machine algorithm

Contents

1	Intro	3
1.1	Raw data files	3
1.2	Amino acid's properties	3
2	Representation and modelization of data	3
2.1	Amino acids and properties	3
2.2	Chains	4
3	Expressions and Formulas	4
3.1	Entropies	4
3.2	Correlation	4
4	Metropolis	5
4.1	Simulation Workflow: From Batch to Single Run	5
4.1.1	The <code>mega.metropolis</code> Manager	5
4.1.2	The <code>run.metropolis</code> Engine	5
4.2	Simulated Annealing Schedule (β)	6
4.3	Energy Function	6
4.4	Model Parameters	7
5	Support Vector Machine	8
5.1	The kernel	8
5.2	Decision function	8
6	SVM results	9
6.1	Validation	10
6.2	Cross-validation of standard kernels	11
6.3	Custom piecewise kernels	13
7	Logistic regression	15
7.1	10-dimensional representation	15
7.2	Model and training	15
7.3	Weights and feature importance	16
7.4	Results	16
7.5	Permutation importance	17
7.6	Comparison with decision tree (CART)	17
7.7	Accuracy and certainty frontier	18
Appendix		19
	SVM formula deduction	19
	SVM simulated annealing	20
	SVM decision function	21
	SVM kernel function comparison	22
	Piecewise kernels performance	23
	Permutation importance	23
	Complete ten-kernel correlation heatmap	24

1 Intro

The aim of this GitHub repository is to delve deeper into biophysics, using simulated annealing and science data to humanize murine antibodies.

We use a frequentist analysis of the given data in `learn_human.txt` and `learn_mouse.txt` in order to measure entropies and correlations.

With those outputs we intend to fine-tune the simulated annealing algorithm for the humanization.

1.1 Raw data files

The `learn_human.txt` and `learn_mouse.txt` data entries are chains of 298 amino acids each. Every amino acid is represented by a character in this string: `-ACDEFGHIKLMNPQRSTVWY`. The `-` is a gap in the chain, as used in the notation convention.

1.2 Amino acid's properties

Every amino acid is a certain chemical compound. Thus, it has chemical properties, which vary from amino acid to amino acid. The used properties in this project are (in order): property-gap, hydrophobic, aromatic, aliphatic, polar, small, minuscule, positively charged, and negatively charged. Every amino acid has assigned at least one property, so for consistency we added the property version of the gap: property-gap, denoted as `-(PROP)`.

2 Representation and modelization of data

2.1 Amino acids and properties

Every amino acid will be simulated/coded as a 21-dimensional vector of real numbers between 0 and 1 juxtaposed to a 9-dimensional vector of real numbers between 1 and 0. That is, some amino acid in our amino acid space, $a \in \mathcal{A}$, will be $a = v \times w \in [0, 1]^{21} \times [0, 1]^9 = \mathcal{A} \subset \mathbb{R}^{21} \times \mathbb{R}^9$.

- The first list will be a 21-dimensional vector, whose i -th component corresponds to the frequency of the i -th amino acid from the aforementioned amino acid string.
- The second list will be a 9-dimensional vector, whose i -th component corresponds to the frequency of the i -th property from the aforementioned property list.

As you can see, these variables are context-dependent, since there is no reference for the frequency of amino acids or properties. Therefore, if it is one that represents an actual amino acid, all frequencies may be 0 but one of them, which should be 1. Same will happen for the property vector. We will call this a *concrete amino acid*.

Example: Let's encode the amino acid cysteine (`C`). It is the third in the amino acid list, thus, the first vector should be $v = (0, 0, 1, 0, \dots, 0)$. Cysteine is hydrophobic and small, then: $w = (0, 1, 0, 0, 0, 1, 0, 0, 0)$, since hydrophobic is in the 2nd position and small in the 6th.

Now changing the context, another setting we will be using is the so called *Schrödinger's amino acid*. That will be the average amino acid, whose elements in both vectors will be *separately* the frequencies of every appearing amino acid.

Separately means that the sum of the components of the first list is 1 and the sum of the components of the second list will also be 1.^[1] More specifically: $\sum_{i=1}^{21} v^i = 1 \wedge \sum_{i=1}^9 w^i = 1$.^[2]

¹This is true for every kind of amino acid, whether it is concrete or Schrödinger's.

²Using superscript for convenience.

The norm of an amino acid, $a = v \times w \in \mathcal{A}$, is defined as:

$$\|a\| = \sqrt{\sum_{i=1}^{21} (v^i)^2}$$

When treating with concrete amino acids, we know that $\|a\| = 1$, since only one of the components can be 1.

2.2 Chains

A chain is an array of 298 amino acids and it represents a whole antibody. Just as before, we will have concrete (all components must be concrete amino acids) and Schrödinger's chains (the rest).

This is a $298 \times (21 + 9)$ matrix of split row conditioned frequencies. We will name the space of all possible chains the chain space, $\mathcal{C} = \mathcal{A}^{298}$. Thus, $c \in \mathcal{C}$ will have the form $c = (c_1, \dots, c_j, \dots, c_{298})$, where $c_j = v_j \times w_j$ and $v_j = (v_j^1, \dots, v_j^i, \dots, v_j^{21})$, $\forall j \in [1, 298] \subset \mathbb{N}$.

The norm of a chain, $c \in \mathcal{C}$, is defined as:

$$\|c\| = \sqrt{\sum_{i=1}^{298} \|c_i\|^2}$$

If c is a concrete chain, then $\|c\| = \sqrt{298}$.

3 Expressions and Formulas

3.1 Entropies

We quantify the variability of the amino acids in a Schrödinger's chain with entropies, applying its expression to both vectors of every amino acid in that chain. This means the result will be a 298 element list of pairs of entropies.

To compute them as follows:

$$S_{v,j} = - \sum_{i=1}^{21} v_j^i \log_2 v_j^i, \quad S_{w,j} = - \sum_{i=1}^9 w_j^i \log_2 w_j^i, \quad S_j = (S_{v,j}, S_{w,j}) \quad \forall j \in [1, 298] \subset \mathbb{N}$$

That previous one is the Shannon entropy, but we may need to use some others:

- Linear entropy: $s = \sum_i p_i(1 - p_i)$
- Rényi entropy: $s_\alpha = \frac{1}{1-\alpha} \sum_i p_i^\alpha$, $\alpha \in [0, 1)$
- Tsallis entropy: $s_\alpha = \frac{1}{q-1} (1 - \sum_i p_i^\alpha)$, $\alpha \in [0, 1)$

3.2 Correlation

The measure of correlation between positions in a chain used in this project is mutual information. It is an entropy measure of how much information (in bits) we can deduce of one random variable given another one, and vice versa.

Mutual information is given by:

$$I(X; Y) = \sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} P(\{X = x\} \cap \{Y = y\}) \log_2 \frac{P(\{X = x\} \cap \{Y = y\})}{P(\{X = x\})P(\{Y = y\})},$$

where X and Y are discrete random variables that take values, x and y , in $\mathcal{X}$ and $\mathcal{Y}$. Since we can identify our vectors, v and w , with distribution functions, we are able to apply this method.

Note that $I(X; Y) = I(Y; X)$, since the product and intersection are symmetric.

Also,

$$I(X; X) = \sum_{x \in \mathcal{X}} P(\{X = x\}) \log_2 \frac{P(\{X = x\})}{(P(\{X = x\}))^2} = - \sum_{x \in \mathcal{X}} P(\{X = x\}) \log_2 P(\{X = x\}) = S(X),$$

where $S(X)$ is the entropy of the random variable X .

In practice, we will get a 298×298 symmetric matrix for amino acids (v -part of the chain) whose diagonal is the entropies of every position. However, for our correlation indices we would rather have all 1's in the diagonal and the other elements going from 0 to 1. We normalize it as follows:

$$i(X; Y) = \frac{I(X; Y)}{\min\{S(X), S(Y)\}}.$$

When $\min\{S(X), S(Y)\} = 0$, the position is fully conserved and $I(X; Y) = 0$ as well. In this case, the code outputs $i(X; Y) \equiv 0$, by convention.

4 Metropolis

Important information: Metropolis assumes CDRs are located in the same position for all antibodies being analyzed; in order for this to be valid, sequences need to be properly aligned beforehand.

4.1 Simulation Workflow: From Batch to Single Run

Two functions are used for running the simulation: a high-level manager (`mega_metropolis`), and a core simulation engine (`run_metropolis`). This design allows for parallelized execution of simulations on multiple initial murine sequences.

4.1.1 The `mega_metropolis` Manager

`mega_metropolis`'s primary responsibilities are:

1. **Data Loading:** It begins by reading a list of initial murine sequences (which we call seeds) from a specified file and saves them for later use.
2. **Batch Management:** It creates a timestamped main directory for the entire simulation batch (format `results/metropolis/YYYY-MM-DD_HH_MM_SS..../`). .
3. **Parallel Execution:** It iterates through each seed sequence using an OpenMP parallel loop. For each seed, it creates a dedicated subdirectory (e.g., `seq_1`, `seq_2`).
4. **Replication:** For each seed, it calls the `run_metropolis` function `n_metropolis` times. This allows for multiple independent simulation runs starting from the same seed, providing statistical significance to the results. The output of each run is saved to a separate file (e.g., `run_1.txt`, `run_2.txt`) within the corresponding seed's subdirectory.

4.1.2 The `run_metropolis` Engine

The `run_metropolis` function performs a single, complete simulated annealing run on one given sequence:

1. **Initialization:** It takes an initial sequence, the human Schrödinger chain, and the pre-calculated matrix of β values.

2. **Simulated Annealing Loop:** It iterates through each row of the β matrix, representing a step in the “cooling” schedule.
3. **Metropolis Sweeps:** For each β^i step, it performs a specified number of Metropolis sweeps (`n_sweeps`). A single sweep involves attempting a mutation at every position in the sequence.
4. **State Acceptance:** We begin by proposing a new state, selecting a new AA (including the gap character) uniformly from the 21 possibilities (ensuring it’s different from the current one). Each proposed mutation is accepted or rejected based on the change in total energy (ΔE) and the current, position-specific, β_j^i value, following the Metropolis criterion: accept if $\Delta E < 0$ or if a random number is less than $e^{-\beta_j^i \Delta E}$.
5. **Data Logging:** After the simulation is complete, it calls a dedicated function to write a detailed report file containing the initial parameters, the final sequence and energy for each β step, and overall acceptance statistics.

4.2 Simulated Annealing Schedule (β)

The β -values, which are inverse of the temperature ($1/k_B T$), are not constant; instead, they are defined through a mechanism, which we call the “engine”:

- **Engine:**

$$\beta_i^j = k_i \cdot \frac{\alpha}{S_j + \varepsilon}, \quad k_i = (c_r)^i \quad \forall i \in [1, N_\beta], j \in [1, L_{\text{chain}}]$$

where c_r is the cooling rate, ε avoids division by 0, α is a scale factor, and S_j is entropy for position j in human reference Schrödinger chain.

4.3 Energy Function

The total energy, E_{total} , is a weighted sum of three components, which we aim to minimize:

$$E_{\text{total}} = w_{\text{log}} E_{\text{log}} + w_{\text{prop}} E_{\text{prop}} + w_{\text{penalty}} E_{\text{penalty}}$$

The weights for each component (`WEIGHT_LOG`, `WEIGHT_PROP`, `WEIGHT_PENALTY`) are defined in ‘head.h’.

1. **Log Humanness Energy, E_{log} :** This term quantifies how ”human-like” a sequence is based on AA frequencies from a reference set of human sequences (the Schrödinger chain):

$$E_{\text{log}} = - \sum_{i=1}^{L_{\text{chain}}} \log(p_i(a_i))$$

where $p_i(a_i)$ is the frequency of the amino acid a_i at position i in the human reference. A large penalty (`ZERO_FREQ_PENALTY_LOG`) is added for amino acids with a frequency below a small threshold (`EPSILON`).

2. **Property Distance Energy, E_{prop} :** This term measures the similarity between the properties of the sequence and the human reference; it’s calculated as the squared Euclidean distance in the property space for each position:

$$E_{\text{prop}} = \sum_{i=1}^{L_{\text{chain}}} \sum_{j=1}^{N_{\text{props}}} \left(\text{prop}_j(a_i) - \overline{\text{prop}_j(i)} \right)^2$$

where $\text{prop}_j(a_i)$ is the value of property j for amino acid a_i , and $\overline{\text{prop}_j(i)}$ is the average value of property j at position i in the Schrödinger chain.

3. **Wandering Penalty Energy, E_{penalty} :** This term discourages the sequence from deviating too far from the original murine sequence; we thus only incentivize mutation if there’s a relatively big gain in humanness. It consists of a penalty based on property distance and a (for now) constant penalty for any mutation:

$$E_{\text{penalty}} = \sum_{i=1}^{L_{\text{chain}}} \left[\left(\sum_{j=1}^{N_{\text{props}}} (\text{prop}_j(a_i) - \text{prop}_j(a_i^{\text{original}}))^2 \right) + P \cdot \delta_{a_i}^{a_i^{\text{original}}} \right]$$

where a_i^{original} is the amino acid at position i in the original murine sequence, P is the mutation penalty (AA_MUTATION_PENALTY), and δ is the “generalized Kronecker delta” (logical if gate). This approach ensures each mutation has a minimum energy cost, while heavily penalizing big property changes.

4.4 Model Parameters

The following table summarizes the key parameters for metropolis in our simulations. These values are defined in `main.c` and `head.h`.

Parameter	Value ^[3]	Description
<code>n_betas</code>	50	Number of cooling steps in the annealing schedule.
<code>n_sweeps</code>	800	Number of Metropolis sweeps per β step.
<code>n_metropolis</code>	20	Number of independent runs per murine seed sequence.
<code>w_log</code>	0.4 (<code>head.h</code>)	Weight for the log-humanness energy component.
<code>w_prop</code>	0.4 (<code>head.h</code>)	Weight for the property-distance energy component.
<code>w_penalty</code>	0.2 (<code>head.h</code>)	Weight for the wandering penalty energy component.
<code>cooling_rate</code> (c_r)	1.02	Multiplicative factor for increasing k_i at each step.
<code>scale_factor</code> (α)	1.0	Global scaling factor for all β values.
<code>AA_MUTATION_PENALTY</code> (P)	1.5 (<code>head.h</code>)	Constant penalty added for any AA mutation away from the original.

³Provisional values.

5 Support Vector Machine

We intend to back up the humanness of the chain outputted by the simulated annealing algorithm with this binary-output method.

The main point of an SVM (abbreviation of Support Vector Machine) is to transform our data space into some other one in such a way that we can separate the data points with a simple hyperplane, a linear classifier.

We will represent each point with a n -dimensional vector, where n is the number of data points. The function that gives us those coordinates is called the kernel.

5.1 The kernel

Our kernel, $k : \mathcal{C} \times \mathcal{C} \rightarrow \mathbb{R}$, will be the Gaussian radial basis function:

$$k(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2\right), \quad \gamma > 0,$$

where $\{\mathbf{x}_i\}$ are the data points.

After applying the kernel we have a matrix $M \in \mathbb{R}^{n \times n}$, the Gram matrix, with the coordinates of our data points in the new space. That will be: $[M]_{i,j} = k(\mathbf{x}_i, \mathbf{x}_j)$.

We also have available four other standard kernel functions:^[4]

- Sigmoid function:

$$k(\mathbf{x}_i, \mathbf{x}_j) = \tanh \frac{\langle \mathbf{x}_i | \mathbf{x}_j \rangle}{298}$$

- Linear function:

$$k(\mathbf{x}_i, \mathbf{x}_j) = \frac{\langle \mathbf{x}_i | \mathbf{x}_j \rangle}{298}$$

- Tanimoto function:

$$k(\mathbf{x}_i, \mathbf{x}_j) = \frac{\langle \mathbf{x}_i | \mathbf{x}_j \rangle}{596 - \langle \mathbf{x}_i | \mathbf{x}_j \rangle}$$

- Cauchy function:

$$k(\mathbf{x}_i, \mathbf{x}_j) = \frac{298}{298 + \|\mathbf{x}_i - \mathbf{x}_j\|^2}$$

The Gram matrix and the kernel are the tools we need in order to craft the decision function.

5.2 Decision function

Using the lagrange multipliers method for minimization^[5], we get the decision function, $f : \mathcal{C} \rightarrow \mathbb{R}$:

$$f(\mathbf{x}) = b + \sum_{i=1}^n \lambda_i s_i k(\mathbf{x}_i, \mathbf{x}),$$

where λ_i are the lagrange multipliers obtained through said method, s_i is the *tag* of the i -th data point (may be +1, when human, or -1, when murine) and b is the bias, obtained as follows:

$$b = s_p - \sum_{i=1}^n \lambda_i s_i k(\mathbf{x}_i, \mathbf{x}_p),$$

where the p -th data point is a support one, that is, the closest to the plane (or any of them) and such that $\lambda_p > 0$.

⁴These constants (298 and $596 = 2 \cdot 298$) that are scattered around the functions seem to be arbitrary, but the written functions are already optimized for our use case: they are normalized and centered so that every output is positive and does not surpass 1.

⁵See [appendix for full derivation](#).

6 SVM results

Using the `learn_human`, `learn_mouse`, `test_human`, and `test_mouse` sets for training^[6] and testing, we get the following results. In this histogram, we plot the distribution of our sign-corrected certainty degree, $s \cdot f(\mathbf{x})$ (positive when the chain is tagged correctly, and negative when misclassified):

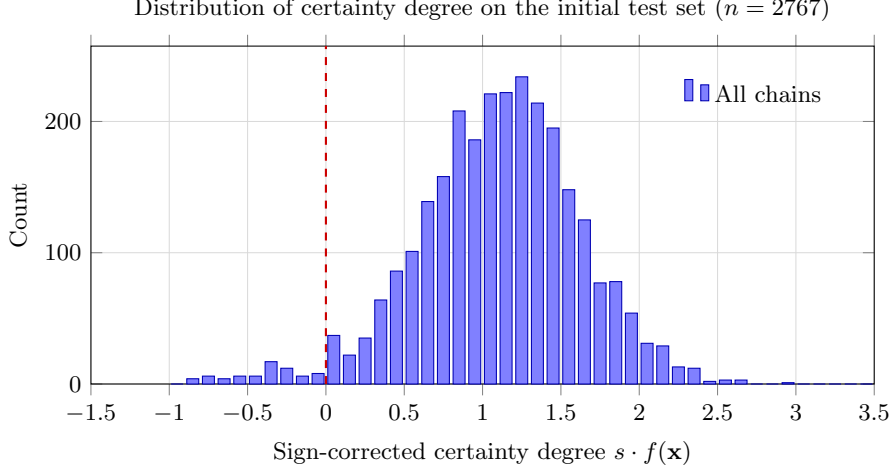


Figure 1: Count with respect to sign-corrected certainty degree on the initial dataset.

This gets us to the confusion matrix:

	Human	Murine	Overall percentages	Human	Murine
Tagged as human	1360	41	Tagged as human	49.15%	1.48%
Tagged as murine	28	1338	Tagged as murine	1.01%	48.36%

Counting human as positive, this gets us to a F1^[7] of ~ 0.9753 ^[8]. On the other hand, being murine the positives, we have an F1 of ~ 0.9749 ^[9]. This takes us to an overall accuracy^[10] of ~ 0.9751 ^[11].

However, those tests were taken with a completely heuristic upper limit, we will call λ_t , for the λ vector. Since we cannot determine λ_t analytically, we must adjust it as a hyperparameter with a validation data set.

⁶Here, *training* refers to computing the Gram matrix.

⁷ $F1 = \frac{2 \cdot TP}{2 \cdot TP + FN + FP}$

⁸Precisely: $F1_H = 0.975259949802797$.

⁹Precisely: $F1_M = 0.974863387978142$.

¹⁰ $Acc = \frac{TP + TN}{TP + TN + FP + FN}$

¹¹Precisely: $Acc = 0.975063245392121$.

6.1 Validation

We have merged all data sets and split them into **learn**, **validation**, and **test**, with 70%, 15%, and 15% of the chains respectively.

Through this method we get an optimal value for $\lambda_t = 10.0$, having tried with powers of 10 from 0.01 to 1000. Using that hyperparameter, we get the following confusion matrix

	Human	Murine	Overall percentages	Human	Murine
Tagged as human	404	4	Tagged as human	60.48%	0.60%
Tagged as murine	6	254	Tagged as murine	0.90%	38.02%

And F1 values and accuracy:

$$F1_H = 0.9878^{[12]} \quad F1_M = 0.9788^{[13]} \quad Acc = 0.9836^{[14]}$$

The following histogram shows the distribution of the sign-corrected certainty degree, as before positive when correct and negative when incorrect tagging:

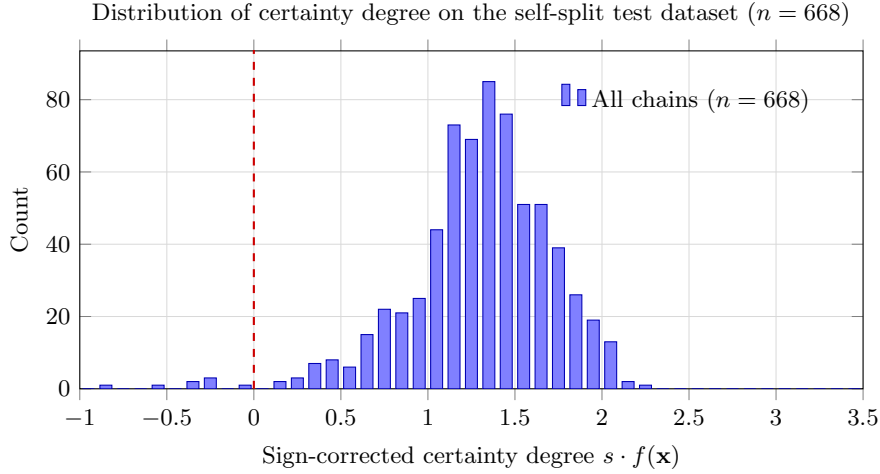


Figure 2: Count with respect to sign-corrected certainty degree using self-split learn, validation, and test datasets.

¹²Precisely: $F1_H = 0.987775061124694$.

¹³Precisely: $F1_M = 0.978805394990366$.

¹⁴Precisely: $Acc = 0.98355754857997$

6.2 Cross-validation of standard kernels

If we now test the other four standard kernels, the resulting distributions look very similar:

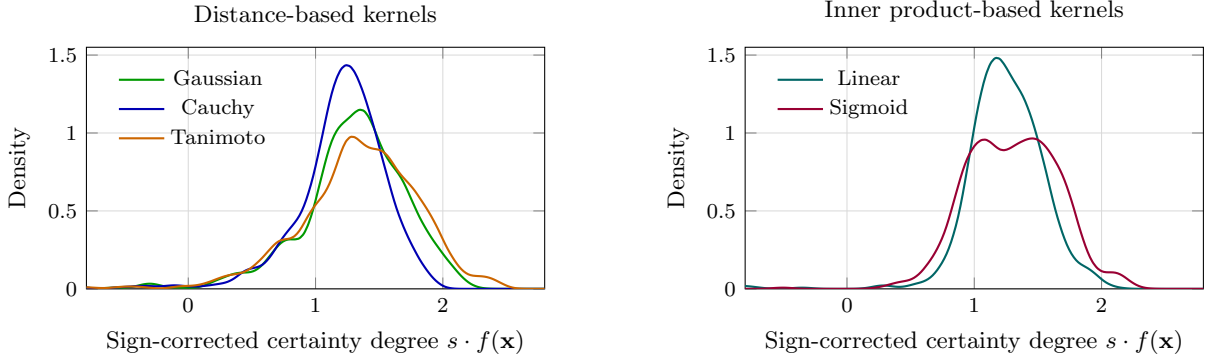


Figure 3: Density distributions of the sign-corrected certainty degree on the test dataset for distance-based kernels (left) and inner product-based kernels (right). Values $s \cdot f > 0$ correspond to correct predictions, while $s \cdot f < 0$ represent errors.

To see how their predictions relate to each other, we plot the scatterplot matrix of decision function outputs $f(\mathbf{x})$ in Figure 4.

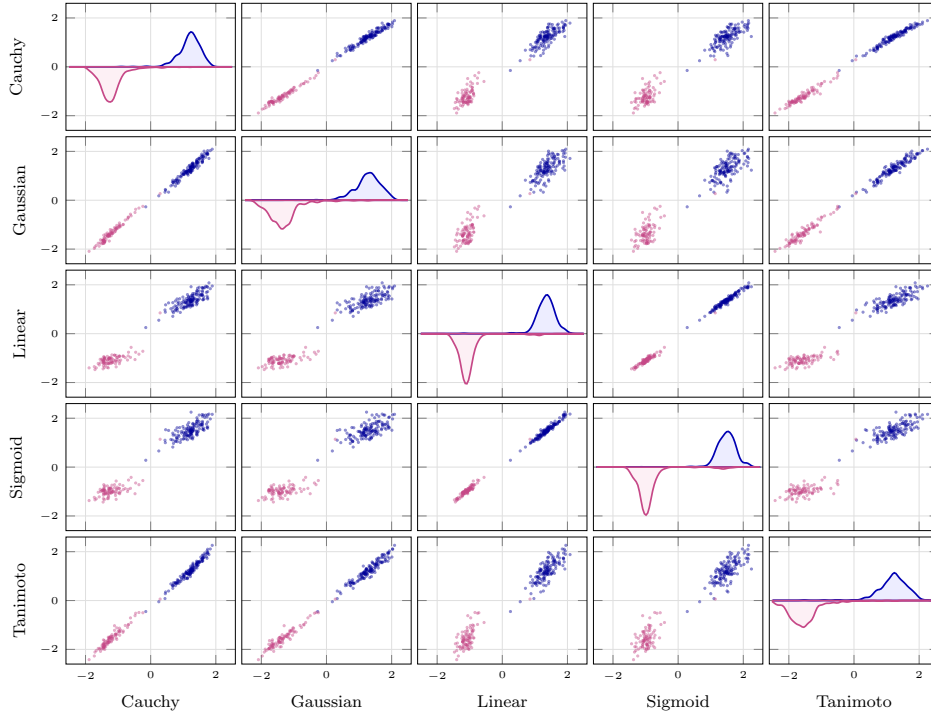


Figure 4: Scatterplot matrix of decision function outputs $f(\mathbf{x})$ across the five standard kernels (human in blue, murine in pink). On the diagonals, human density is plotted upward and murine downward to avoid overlap and keep all axes on the same scale.

Here we have the correlation matrix for the five standard kernels:^[15]

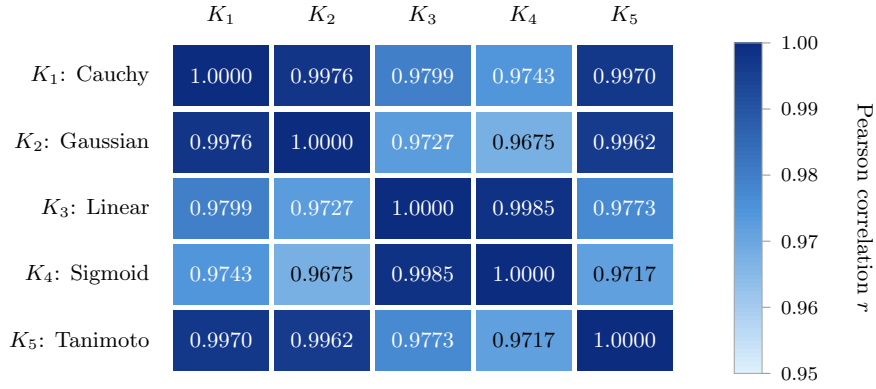


Figure 5: Correlation heatmap across the raw decision function outputs $f(\mathbf{x})$ of the five standard kernels on the test dataset.

As we can see from both the scatterplot matrix and the correlation table, the five standard kernels are very correlated ($r > 0.96$ for all pairs, and $r \approx 0.998$ between Linear and Sigmoid). Because these functions are smooth across the whole range $[0, 1]$, they apply a uniform scale and do not focus on the narrow similarity interval where antibodies switch from murine to human.

¹⁵The correlations were expected, to a certain level, because of how similar the functions used as kernel are to each other. See appendix for detail.

6.3 Custom piecewise kernels

Because the standard kernels scale smoothly over the entire $[0, 1]$ interval, they treat all similarity values uniformly and do not give special weight to the narrow region where murine antibodies switch to human. To focus directly on this decision boundary, we designed a custom piecewise linear kernel with a steep ramp of width W centered at C :

$$k(\mathbf{x}_i, \mathbf{x}_j) = \begin{cases} \frac{L}{C - \frac{W}{2}} x, & x < C - \frac{W}{2} \\ \frac{H - L}{W} \left(x - C + \frac{W}{2} \right) + L, & |x - C| \leq \frac{W}{2} \\ \frac{1 - H}{1 - C - \frac{W}{2}} \left(x - C - \frac{W}{2} \right) + H, & x > C + \frac{W}{2} \end{cases}$$

where $x = \frac{\langle \mathbf{x}_i | \mathbf{x}_j \rangle}{298} \in [0, 1]$, with baseline levels $L = 0.2$ and $H = 0.8$.

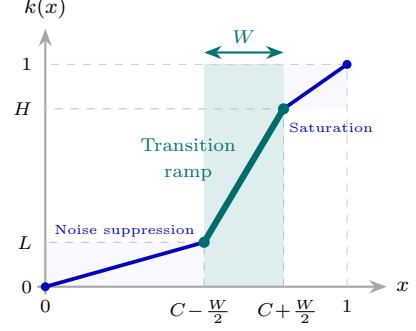


Figure 6: Mathematical definition and schematic mapping of the custom piecewise linear kernel. The narrow transition ramp of width W centered at C magnifies small differences right around the threshold, while sequences far from the boundary are flattened into constant baselines.

By choosing a narrow width ($W = 0.08$) and placing the center C in the transition region ($C \in \{0.78, 0.80, 0.82, 0.84\}$), small differences between sequences near the threshold produce large changes in the kernel, while sequences far from the boundary stay in constant regions.

To measure both classification accuracy and margin certainty in a single score between 0 and 1, we define the quadratic certainty score $\mathcal{Q}$:

$$\mathcal{Q} = 1 - \frac{1}{n} \sum_{i=1}^n \left(1 - \frac{1}{1 + e^{-s_i f(\mathbf{x}_i)}} \right)^2 \in [0, 1]$$

where $s_i \in \{-1, +1\}$ is the true tag and $f(\mathbf{x}_i)$ is the decision function output. Intuitively, confident correct predictions yield values close to 1.0, predictions right on the boundary give 0.75, and confident wrong predictions drop close to 0.0.

Testing these custom kernels on the test dataset yields the results shown in Figure 7 (see appendix for the full table):

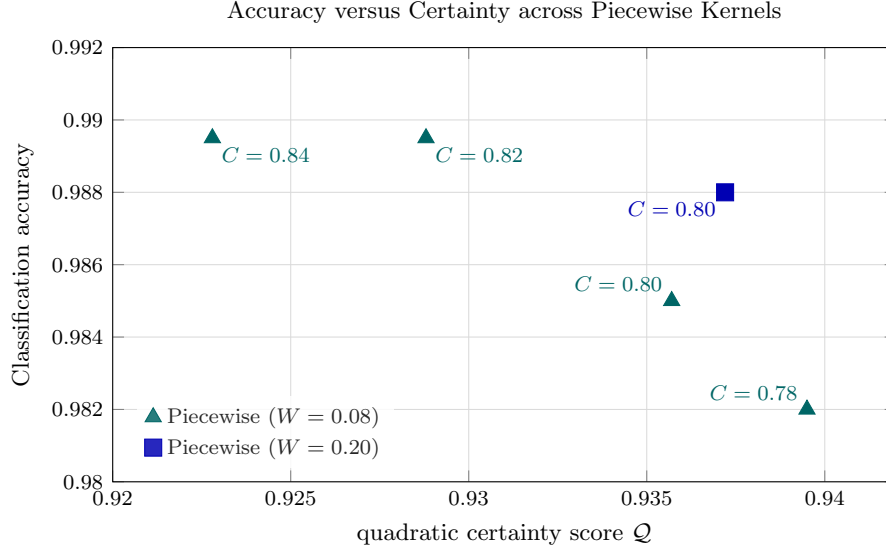


Figure 7: Classification accuracy versus quadratic certainty score Q for the custom piecewise kernels on the test dataset (full values in appendix).

The piecewise kernels with $C = 0.82$ and $C = 0.84$ reach an $F1_H$ of 0.9915 and 98.95% accuracy, higher than any of the five standard kernels.

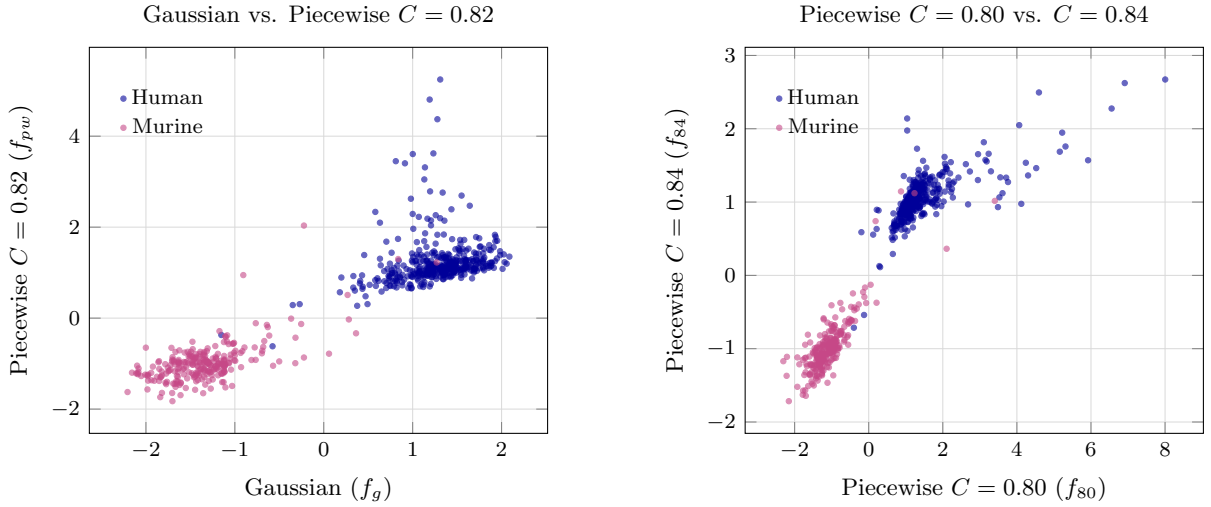


Figure 8: Scatterplots of decision output for custom piecewise kernels compared against standard Gaussian (left) and between two fine-tuned piecewise centers (right).

This leaves us with 10 distinct kernel tests: 5 standard global kernels and 5 localized piecewise kernels. In the next section, we train a logistic regression model to find the best linear combination of all 10 kernel outputs.

7 Logistic regression

Now that we have computed the decision functions for several kernels, we can combine their outputs. Instead of just taking the binary sign of $f(\mathbf{x})$, we use their continuous values directly.

7.1 10-dimensional representation

For each antibody chain $\mathbf{x}$, we build a 10-dimensional vector from the outputs of our 10 SVM kernel tests:

$$\mathbf{v}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_{10}(\mathbf{x})) \in \mathbb{R}^{10},$$

where the components correspond to:

- f_1 : Cauchy
- f_2 : Gaussian (RBF)
- f_3 : Linear
- f_4 : Sigmoid
- f_5 : Tanimoto
- f_6 : Piecewise ($C = 0.80, W = 0.20$)
- f_7 : Piecewise ($C = 0.80, W = 0.08$)
- f_8 : Piecewise ($C = 0.82, W = 0.08$)
- f_9 : Piecewise ($C = 0.84, W = 0.08$)
- f_{10} : Piecewise ($C = 0.78, W = 0.08$)

Before training, we standardize every feature using the mean μ_k and standard deviation σ_k measured on the validation set:

$$\hat{f}_k(\mathbf{x}) = \frac{f_k(\mathbf{x}) - \mu_k}{\sigma_k}.$$

7.2 Model and training

We take a linear combination of the standardized outputs:

$$z(\mathbf{x}) = b + \sum_{k=1}^{10} w_k \hat{f}_k(\mathbf{x}),$$

where w_k is the weight for the k -th kernel and b is the bias. The probability of being human is given by the usual logistic function:

$$P(\text{Human} \mid \mathbf{x}) = \frac{1}{1 + e^{-z(\mathbf{x})}}.$$

If $P(\text{Human} \mid \mathbf{x}) \geq 0.5$, we classify the chain as human (+1), and murine (−1) otherwise.

We train the model on the validation dataset (667 samples) with gradient descent, minimizing the binary cross-entropy loss with an L2 weight penalty:

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right] + \frac{\lambda}{2} \sum_{k=1}^{10} w_k^2,$$

where $y_i \in \{0, 1\}$, $\hat{y}_i = P(\text{Human} \mid \mathbf{x}_i)$, $\lambda = 0.01$, and learning rate $\eta = 0.5$ over 2000 iterations.

7.3 Weights and feature importance

After training, all learned weights turn out to be positive, meaning that higher scores across all kernels consistently push the prediction towards human. In the following plot, we show the learned weights in descending order:

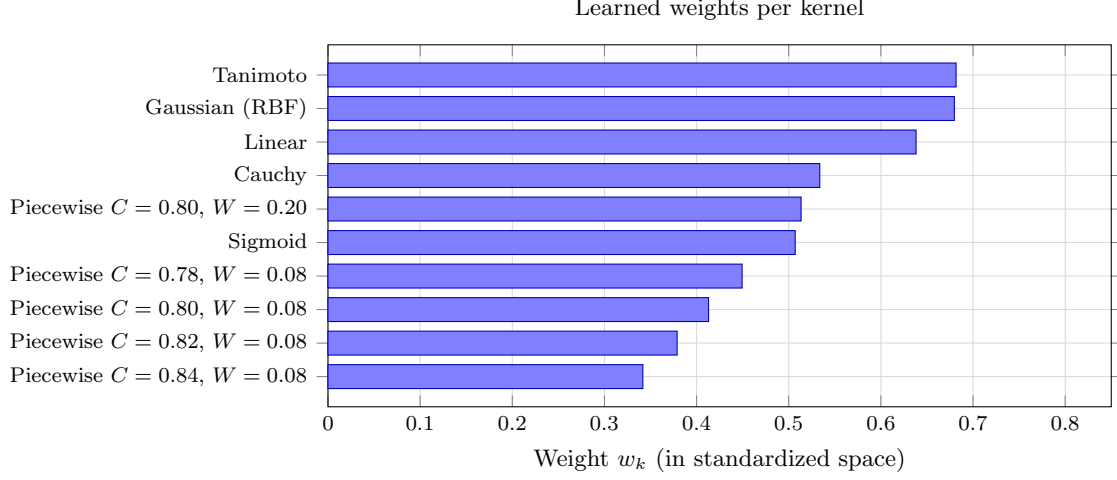


Figure 9: Learned logistic regression weights in descending order.

Tanimoto ($w \approx 0.6816$), Gaussian ($w \approx 0.6797$), and Linear ($w \approx 0.6382$) carry the largest weights, while the narrower piecewise kernels ($W = 0.08$) act as fine-tuning corrections right around the decision threshold.

7.4 Results

Evaluating the trained model on both datasets gives the following confusion matrices:

	Human	Murine		Human	Murine
Tagged as human	403	2	Tagged as human	403	4
Tagged as murine	2	260	Tagged as murine	5	256
Validation dataset (667 samples)			Test dataset (668 samples)		

On the validation dataset:

$$F1_H = 0.9951^{[16]} \quad F1_M = 0.9924^{[17]} \quad Acc = 0.9940, \quad Q = 0.9927$$

And on the test dataset:

$$F1_H = 0.9890^{[18]} \quad F1_M = 0.9827^{[19]} \quad Acc = 0.9865, \quad Q = 0.9850$$

¹⁶Precisely: $F1_H = 0.995061728395062$.

¹⁷Precisely: $F1_M = 0.992366412213740$.

¹⁸Precisely: $F1_H = 0.988957055214724$.

¹⁹Precisely: $F1_M = 0.982725527831094$.

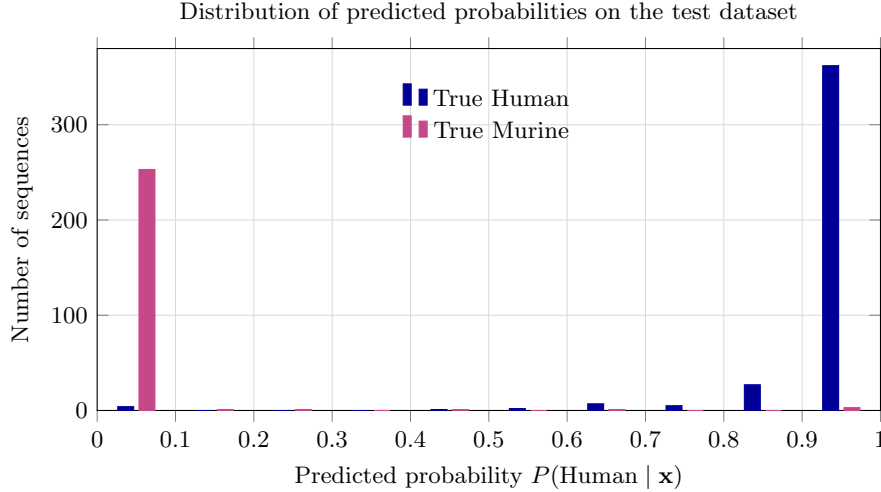


Figure 10: Distribution of predicted human probabilities $P(\text{Human} \mid \mathbf{x})$ on the test dataset (408 human, 260 murine). Most predictions concentrate near 0 and 1, with very few ambiguous cases around 0.5.

7.5 Permutation importance

Because our kernels are strongly correlated (as shown in the heatmap), their regression weights are shared and can somewhat mask each kernel’s individual contribution. To isolate how much predictive power each kernel brings on its own, we compute the permutation importance on the test dataset: for each kernel, we randomly shuffle its values across the 668 test chains 500 times and measure the resulting drop in accuracy and F1 scores.

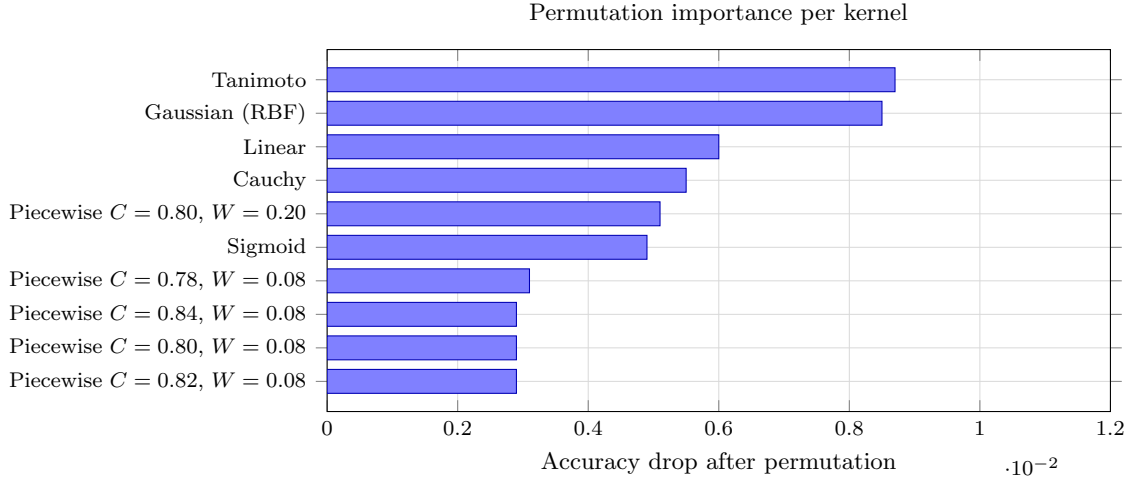


Figure 11: Drop in test accuracy when each kernel is randomly shuffled 500 times. [See appendix for complete table with F1 drops.](#)

Shuffling any of the ten kernels causes a noticeable drop in accuracy, confirming that each one contributes useful information to the final decision. In line with the learned weights, Tanimoto and Gaussian have the largest individual impact.

7.6 Comparison with decision tree (CART)

As a baseline comparison, we also trained a decision tree (CART, depth 5) on the exact same 10-dimensional feature vectors. The results on the test set are summarized in Table 1:

Model	Accuracy	F1 Human	F1 Murine	$\mathcal{Q}$
CART (depth 5)	0.9656	0.9714	0.9568	0.9131
Logistic regression	0.9865	0.9890	0.9827	0.9850

Table 1: Test dataset results comparing the decision tree and logistic regression models.

The decision tree is held back by its rigid, axis-aligned cuts in the 10-dimensional space, scoring noticeably lower across all metrics ($\mathcal{Q} = 0.9131$). In contrast, logistic regression combines the continuous margins linearly into a smooth decision boundary, reaching an accuracy of 98.65% and an $F1_H$ of 0.9890.

7.7 Accuracy and certainty frontier

To put everything into perspective, we plot the test classification accuracy against our quadratic certainty score $\mathcal{Q}$ for all individual kernels and the logistic regression model in Figure 12:^[20]

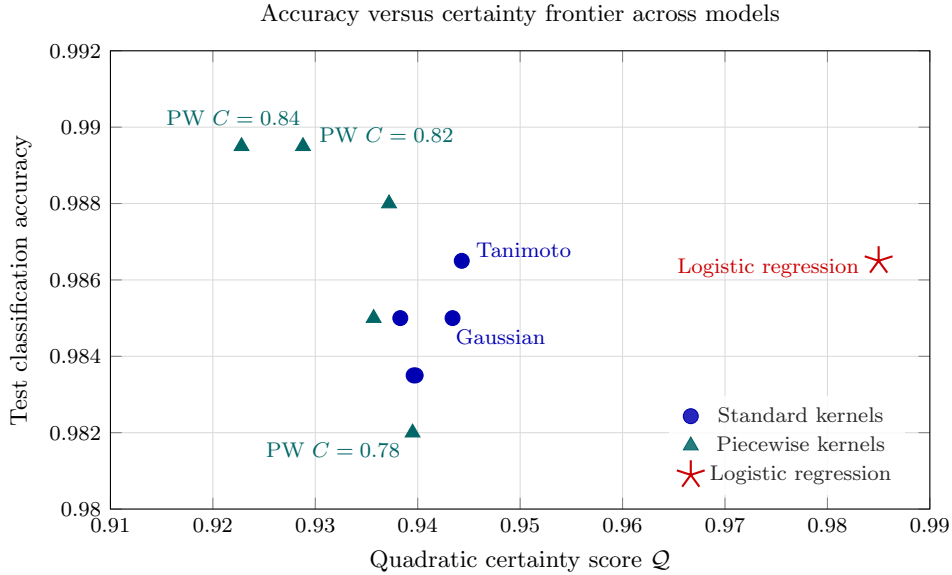


Figure 12: Test classification accuracy versus quadratic certainty score $\mathcal{Q}$ across standard kernels (blue circles), piecewise kernels (teal triangles), and the logistic regression model (red star).

This frontier highlights the main trade-off: while isolated piecewise kernels achieve slightly higher raw accuracy (98.95%) at the expense of certainty, and standard kernels max out around $\mathcal{Q} \approx 0.944$, our logistic regression model achieves the best of both worlds. It maintains a high accuracy of 98.65% while dramatically pushing certainty up to $\mathcal{Q} = 0.9850$, outperforming every single standalone kernel in margin confidence.

²⁰CART is omitted from the plot since its performance sits well below the axis bounds ($\text{Acc} \approx 0.9656$, $\mathcal{Q} \approx 0.9131$).

Appendix

SVM formula deduction

We have a cloud of points in a higher-dimensional space and we want a plane that separates the ones marked human and the ones marked as murine^[21]. But we do not only want *one* that does that, we need the plane that does that the best: that is the one that maximizes the distance to the closest points.

We *must* assume that the two groups of points are separable by a plane in our space and let us say the distance^[22] between the two clouds is $2d$; we will need this later.

Our solution will have the form:

$$\mathbf{w}^T \mathbf{X} + b = 0,$$

where $\mathbf{w}$ is the vector perpendicular to the solution plane, $\mathbf{X}$ is a variable-coordinate vector and b is a real number we will call bias.

Since the norm of $\mathbf{w}$ is arbitrary, we can set $\|\mathbf{w}\| = 1/d$. Thus, any of our data points inputted in the function $\mathbf{x} \mapsto |\mathbf{w}^T \mathbf{x} + b|$ will give an output greater or equal to 1 (the nearest ones will give 1, otherwise strictly greater than 1).^[23]

Furthermore, we know whether the result will be negative and then inverted by the absolute value. The ones such that $\mathbf{w}^T \mathbf{x}_i + b \leq -1$ will have also the negative tag: $s_i = -1$. Thus,

$$(\mathbf{w}^T \mathbf{x}_i + b) s_i \geq 1 \quad \forall i \in \{1, \dots, n\}.$$

Maximizing d is equivalent to minimizing $\|\mathbf{w}\|$, which is again equivalent to minimizing $\frac{1}{2} \|\mathbf{w}\|^2$, this will come handy after some computations.

We have a optimization problem subject to restrictions:

$$\underset{\mathbf{w}, b}{\text{minimize}} \quad \frac{1}{2} \|\mathbf{w}\|^2, \quad \text{subject to } (\mathbf{w}^T \mathbf{x}_i + b) s_i \geq 1 \quad \forall i \in \{1, \dots, n\}$$

We define the Lagrangian with the classification margin:

$$\mathcal{L}(\mathbf{w}, b, \lambda) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \lambda_i (s_i (\mathbf{w}^T \mathbf{x}_i + b) - 1)$$

We set the partial derivatives to 0:

$$\nabla_{\mathbf{w}} \mathcal{L} = 0 \implies \mathbf{w} = \sum_{i=1}^n \lambda_i s_i \mathbf{x}_i \quad \frac{\partial \mathcal{L}}{\partial b} = 0 \implies \sum_{i=1}^n \lambda_i s_i = 0$$

We substitute $\mathbf{w}$ back into the Lagrangian and apply the restriction for b :

$$\mathcal{L}(\lambda_1, \dots, \lambda_n) = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j s_i s_j \mathbf{x}_i^T \mathbf{x}_j - \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j s_i s_j \mathbf{x}_i^T \mathbf{x}_j - b \sum_{i=1}^n \lambda_i s_i + \sum_{i=1}^n \lambda_i$$

We simplify to obtain the dual formulation:

$$\mathcal{L}(\lambda_1, \dots, \lambda_n) = \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j s_i s_j \mathbf{x}_i^T \mathbf{x}_j,$$

²¹Those are what we call tags: +1 for human and -1 for murine chains.

²²Distance from one set to another is defined as the minimum distance from one point of one set to a point of the other one. Those two (or more) points will be called support vectors.

²³Notice that $\mathbf{x}$ and $\mathbf{x}$ are different, in fact, $\mathbf{x}_i$ is the data point $\mathbf{x}_i$ in the transformed space ($\varphi(\mathbf{x}_i) = \mathbf{x}_i$, being φ the transformation of our space such that $\varphi(\mathbf{x}_i) \cdot \varphi(\mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j = k(\mathbf{x}_i, \mathbf{x}_j)$).

We use our kernel notation for the inner products:

$$\mathcal{L}(\lambda_1, \dots, \lambda_n) = \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j s_i s_j k(\mathbf{x}_i, \mathbf{x}_j),$$

$$\text{with } \sum_{i=1}^n \lambda_i s_i = 0 \text{ and } \lambda_i \geq 0 \forall i \in \{1, \dots, n\}$$

This way we find the lagrange multipliers, maximizing the Lagrangian subject to the restrictions given.

Usually, this optimization problem is solved via quadratic programming, which takes advantage of some of the properties of the kernel (like smoothness and uniform concavity, $f''(x) < 0$, with respect to the inner product). However, it will be convenient not to have these constraints for some other kernels we will be crafting. Instead of the common quadratic programming for maximization, we will be using a simulated annealing algorithm (for minimization of the negated function).

SVM simulated annealing

The hard part is out of the way: we have our function to optimize. Now we just need the stochastic change function.

Let's consider the naïve approach. We could write a loop that adds a random increment to every λ_i except the last one, and then forces the constraint to hold by setting that final one to:

$$\lambda_n = -s_n \sum_{i=1}^{n-1} \lambda_i s_i$$

Watch out: for this step to actually be valid, the result must fall between 0 and our upper limit hyperparameter λ_t (the one from the SVM results section). Randomly hitting a valid number this way is incredibly slow and mathematically unlikely.

Instead, we made the algorithm about 20 times faster by picking just two random indices, i and j . We apply a random increment to λ_i and simply adjust λ_j to balance it out and satisfy the constraint. We take advantage of this localized change in our energy function as well. Calculating the product $\mathbf{x}^T G \mathbf{x}$ from scratch to get $\mathcal{L}$ at every single step is computationally expensive and bogs everything down. Since we are only modifying two components of the λ vector, let's not redo all the math. We can keep the last computed energy and just add or subtract the change:

$$\Delta E = -d(1 - s_i s_j) + d s_i (F_i - F_j) + \frac{1}{2} d^2 ([G]_{i,i} + [G]_{j,j} - 2[G]_{i,j})$$

Here, d is our proposed increment $\Delta \lambda_i$, the class labels are $s \in \{-1, 1\}$, G is the kernel Gram matrix, and $F_k = \sum_m \lambda_m s_m [G]_{k,m}$ acts as our running partial evaluation for the k -th sample.

Let's break down exactly where this comes from. Because of the linear constraint $\sum_k \lambda_k s_k = 0$, shifting λ_i by d immediately forces $\Delta \lambda_j$ to be $-d s_i s_j$. We can expand the exact difference

$$\Delta E = E(\lambda + \Delta \lambda) - E(\lambda)$$

from our dual objective energy

$$E(\lambda) = \frac{1}{2} \sum_{k,m} \lambda_k \lambda_m s_k s_m [G]_{k,m} - \sum_k \lambda_k$$

directly into the components of our shortcut equation. First, let's look at the linear portion $-\sum \lambda_k$.

The variation here is simply:

$$-(\Delta\lambda_i + \Delta\lambda_j) = -(d - ds_i s_j) = -d(1 - s_i s_j)$$

Next, consider the cross interaction between the new increments and the existing values of all the other multipliers in the quadratic form. This gives $\Delta\lambda_i s_i F_i + \Delta\lambda_j s_j F_j$. Since $s_j^2 = 1$, it simplifies neatly to $ds_i(F_i - F_j)$. Finally, the increments also interact with themselves. If we expand the quadratic form exclusively for our modified components i and j , we get:

$$\frac{1}{2} [(\Delta\lambda_i s_i)^2 [G]_{i,i} + (\Delta\lambda_j s_j)^2 [G]_{j,j} + 2(\Delta\lambda_i s_i)(\Delta\lambda_j s_j)[G]_{i,j}]$$

which collapses straight into $\frac{1}{2}d^2([G]_{i,i} + [G]_{j,j} - 2[G]_{i,j})$.

By tracking these F values, which only require an $\mathcal{O}(n)$ update whenever we accept a move, we bring the complexity of calculating the energy difference for any proposed stochastic jump crashing down from $\mathcal{O}(n^2)$ to just $\mathcal{O}(1)$. This mathematical shortcut is precisely what makes simulating millions of iterations computationally viable.

SVM decision function

For our decision function we just want to know which side of the plane is on. We achieve that plugging the new data point into our plane equation:

$$\mathbf{w}^T \mathbf{x} + b = \left(\sum_{i=1}^n \lambda_i s_i \mathbf{x}_i^T \right) \mathbf{x} + b = \sum_{i=1}^n \lambda_i s_i \mathbf{x}_i^T \mathbf{x} + b = \sum_{i=1}^n \lambda_i s_i k(\mathbf{x}_i, \mathbf{x}) + b$$

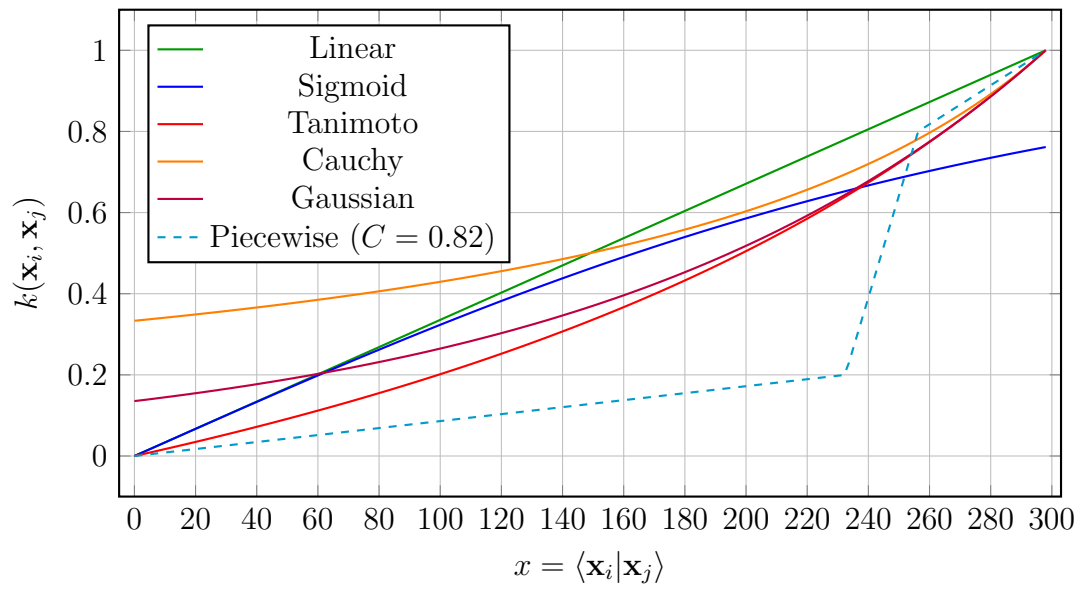
$$f : \mathbf{x} \mapsto b + \sum_{i=1}^n \lambda_i s_i k(\mathbf{x}_i, \mathbf{x})$$

Also, since $(\mathbf{w}^T \mathbf{x}_p + b)s_p = 1$ for some support vector $\mathbf{x}_p$, expanding $\mathbf{w}$:

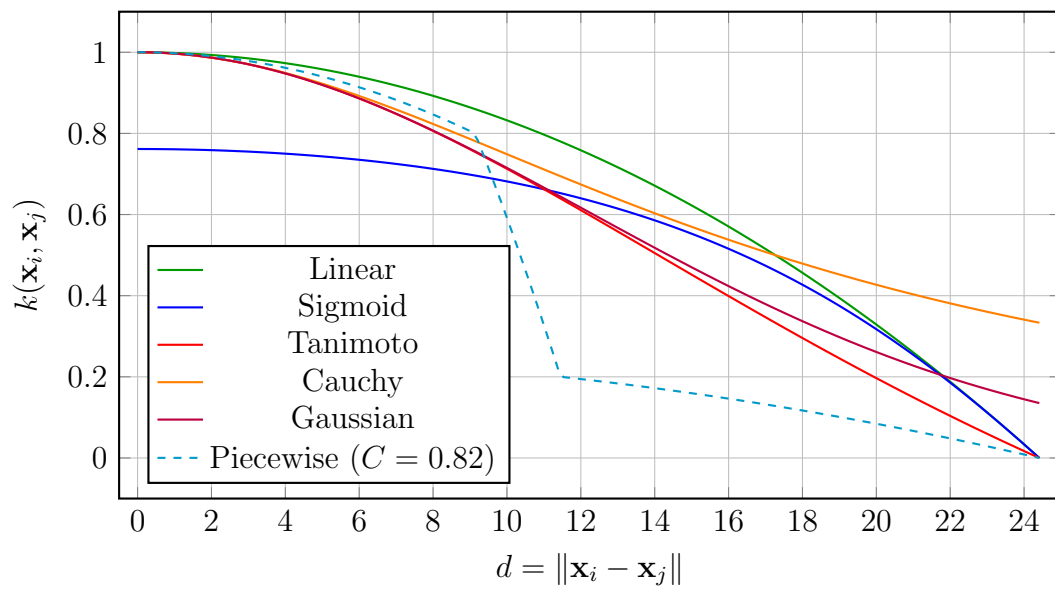
$$b = s_p - \sum_{i=1}^n \lambda_i s_i k(\mathbf{x}_i, \mathbf{x}_p)$$

SVM kernel function comparison

All Kernels vs. Inner Product



All Kernels vs. Distance



Piecewise kernels performance

Kernel parameters	Accuracy	F1 _H	F1 _M	$\mathcal{Q}$
Piecewise ($C = 0.80$, $W = 0.20$)	0.9880	0.9903	0.9844	0.9372
Piecewise ($C = 0.80$, $W = 0.08$)	0.9850	0.9878	0.9806	0.9357
Piecewise ($C = 0.82$, $W = 0.08$)	0.9895	0.9915	0.9865	0.9288
Piecewise ($C = 0.84$, $W = 0.08$)	0.9895	0.9915	0.9865	0.9228
Piecewise ($C = 0.78$, $W = 0.08$)	0.9820	0.9853	0.9769	0.9395

Table 2: Performance of the custom piecewise kernels on the test dataset (668 samples), including the quadratic certainty score $\mathcal{Q}$.

Permutation importance

Kernel	Accuracy drop	F1 _H drop	F1 _M drop
Tanimoto	+0.0087	+0.0072	+0.0109
Gaussian (RBF)	+0.0085	+0.0070	+0.0106
Linear	+0.0060	+0.0050	+0.0075
Cauchy	+0.0055	+0.0046	+0.0069
Piecewise ($C = 0.80$, $W = 0.20$)	+0.0051	+0.0042	+0.0064
Sigmoid	+0.0049	+0.0041	+0.0061
Piecewise ($C = 0.78$, $W = 0.08$)	+0.0031	+0.0026	+0.0039
Piecewise ($C = 0.84$, $W = 0.08$)	+0.0029	+0.0024	+0.0037
Piecewise ($C = 0.80$, $W = 0.08$)	+0.0029	+0.0024	+0.0036
Piecewise ($C = 0.82$, $W = 0.08$)	+0.0029	+0.0024	+0.0036

Table 3: Permutation importance measured on the test dataset across 500 random shuffles per feature.

Complete ten-kernel correlation heatmap

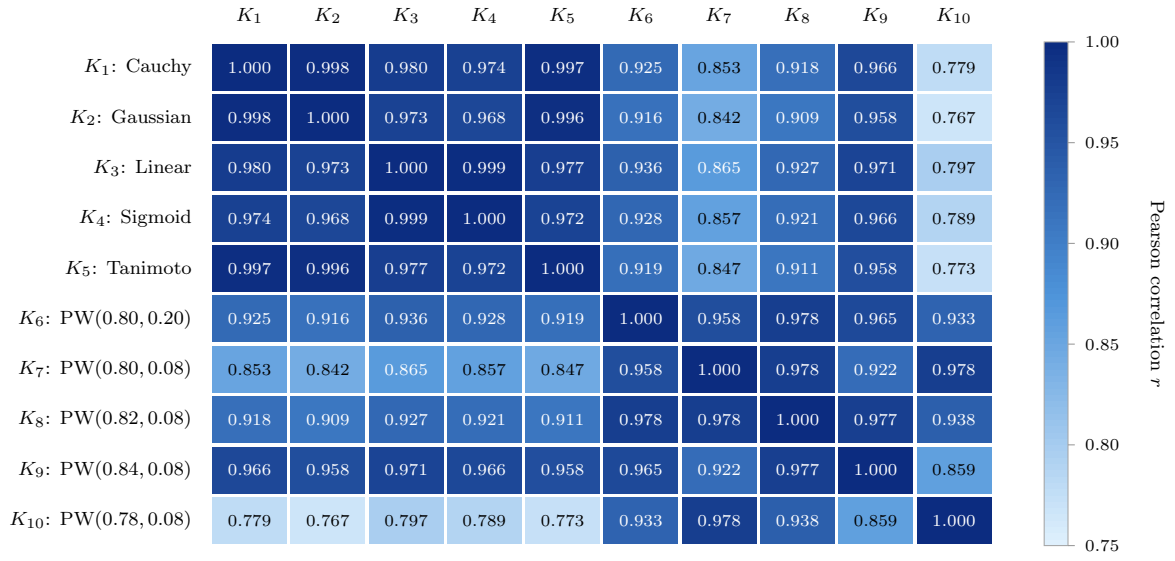


Figure 13: Pearson correlation between the decision function outputs of all ten kernels on the test dataset (668 sequences).